Campus Event Hub

Project documentation for Web Technologies (AIM3140)

Name	Sahil Lenka
Registration no.	2427010391
Programme	B.Tech AIML, Section C, V Semester, Manipal University Jaipur
Course	AIM3140 Web Technologies (project component: backend and frontend)
Live site	https://campus-event-hub-lake.vercel.app
Source code	https://github.com/Sahil-u07/campus-event-hub

1. Aim of the project

Events on a campus are usually announced on posters and in group chats. Students do not know how many seats are left, and hosts do not know how many people will come. Campus Event Hub puts every event in one place. A student can search events, register in one click and, if an event is full, join a waitlist that moves forward by itself. A host can see who registered, tick people off at the door and download the list.

The project is a full web application with a frontend (HTML, CSS, Bootstrap, JavaScript, jQuery), a Python backend (Django), a database and a live deployment.

2. What the application does

Role	Features
Student	Browse upcoming events with a seat meter. Live search without reloading the page. Filter by category and by time (today, this week, later), show only events with free seats, and sort by soonest or most popular. Register or cancel in one click. Join the waitlist when an event is full and see your position. Save events with a heart button, and download an .ics file to add an event to a calendar. A “My events” page lists events you are going to, your waitlist positions and events you host. Dark and light themes.
Host	Create, edit and delete events (only the creator can). Open the attendee dashboard: confirmed list, waitlist, check-in switch for each person, and a CSV download. Raising the capacity moves people up from the waitlist.
Admin	Django admin to manage categories, events, registrations and saved events, with search and filters.

3. Technologies and how they map to the syllabus

Syllabus topic	Where it is used in the project
HTML: tags, lists, tables, links, forms	Templates in events/templates: event cards, attendee tables, sign up, log in and event forms with server-side validation
CSS and Bootstrap grid	events/static/css/site.css on top of Bootstrap 5; responsive grid, navbar, cards, tabs; works from 375 px phones to desktop; dark and light themes
JavaScript: events, functions, DOM	events/static/js/site.js: builds cards, updates seat counts and buttons, theme toggle, toast messages, show-password button
jQuery, JSON and AJAX	Search, filters, load more, register, cancel, save and check-in all call JSON endpoints with jQuery and update the page without a reload
Python for the web, GET and POST	events/views.py: GET for pages and search, POST for forms and actions
MySQL and CRUD	The same code runs on MySQL when DB_NAME is set; sql/schema.sql shows the tables and CRUD queries
Django: MVT, models, ORM, admin, URL routing	models.py, views.py, templates, urls.py, admin.py; all queries use the ORM (filter, annotate, select_related, constraints)

NodeJS and AngularJS are in the syllabus but are not used in this project.

4. Design

The app follows Django’s Model-View-Template pattern: URLs are routed to views, views read and write models through the ORM, and templates build the HTML. Pages work without JavaScript for browsing; jQuery adds the live behaviour. The database has four tables of its own, plus Django’s built-in user table.

Table	Purpose and main fields
Category	Event type (name).
Event	title, description, venue, start time, capacity, category, organiser (the host).
Registration	event, user, status (confirmed or waitlisted), checked_in, created time. A unique pair (event, user) stops one person taking two seats.
SavedEvent	user and event the user saved; unique per pair.

5. How the main parts work

- **Search and filters.** The form sends its values to /api/events/. The server filters, sorts and returns nine events per page as JSON. jQuery rebuilds the cards, updates the result count and writes the filters into the page address, so a filtered view can be shared.
- **Registration and waitlist.** Seats left means capacity minus confirmed registrations. If no seat is free the new registration is saved as waitlisted, in order of arrival. When a confirmed person cancels, or the host raises the capacity, the earliest waitlisted people are confirmed.
- **Last seat.** Register and cancel run inside a database transaction that locks the event row, so two people clicking at the same moment cannot both get the last seat.
- **Host tools.** Only the event's creator can open the attendee page, toggle check-in or download the CSV; anyone else gets a 403 error. In the CSV, names starting with = + - or @ are prefixed so a spreadsheet cannot run them as a formula.
- **Accounts.** Django's authentication handles sign up and login. Passwords are hashed and must pass Django's checks (8 or more characters, not common, not only digits). Form errors appear in a box at the top of the page and next to each field.

6. JSON endpoints

Endpoint	Result
GET /api/events/?q=&category=&when=&available=&sort=&page=	events, total, has_next, page
POST /api/events/<id>/register/ and /cancel/	state (confirmed, waitlisted or none), seats_left, position
POST /api/events/<id>/save/	saved (true or false)
POST /api/registrations/<id>/checkin/	checked_in (host only)

7. Security and testing

- Every POST, including the AJAX ones, carries a CSRF token. Actions that need an account return 401, and owner-only pages return 403 to other users.
- On the live site the cookies are marked Secure, the site runs over HTTPS, and the secret key comes from an environment variable, not from the code.
- **70 automated tests** (python manage.py test) cover registration and waitlist rules, filters and paging, saved events, the calendar file, the host dashboard, CSV escaping, permissions, forms and the login and signup pages. They run on an in-memory SQLite database.

8. Deployment

The code is on GitHub. Every push to the main branch is built and deployed by **Vercel**. Vercel's disk is read-only, so the live site stores data in a **Supabase PostgreSQL** database, selected by the POSTGRES_URL environment variable. The build step in vercel.json runs the migrations and loads sample events. Locally, with no settings, the project uses a SQLite file; setting DB_NAME switches it to MySQL.

9. Running it on a computer

Python 3.10 or newer is needed. In a terminal: clone the repository, create and activate a virtual environment, then run:

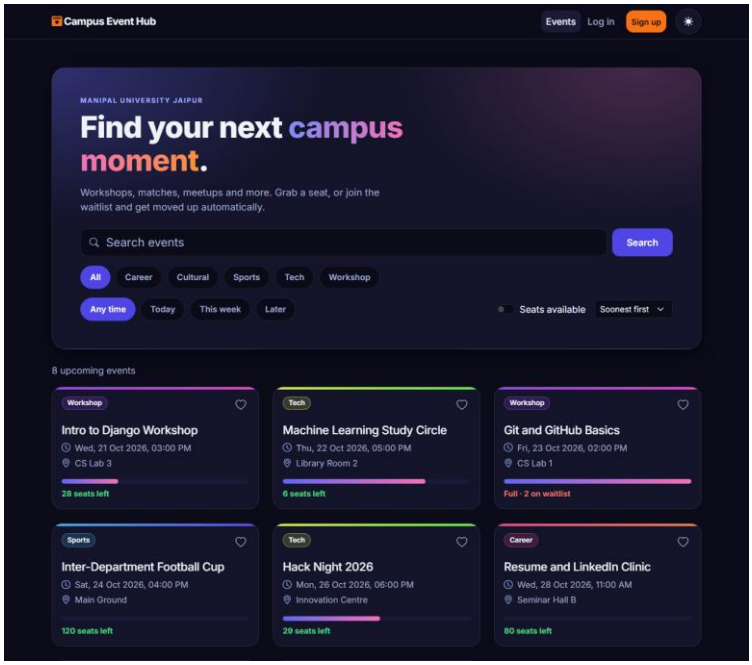
```
pip install -r requirements.txt | python manage.py migrate | python manage.py seed_demo | python manage.py runserver
```

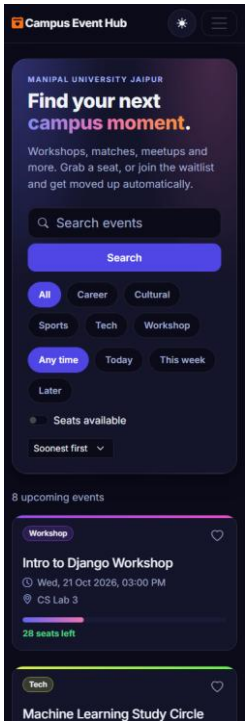
Then open http://127.0.0.1:8000/. Accounts made on the live site and on a local computer are separate, because each has its own database.

10. Limits and future work

- No email or SMS messages, so a waitlist promotion shows on the site only. Notifications would be the next feature.
- Events have no poster images, because the hosting disk cannot store uploads. Cloud storage would fix this.
- The tested setups are SQLite on a computer and PostgreSQL on the live site. MySQL is supported by the code and the schema file but was not part of the test runs.
- Free hosting can be slow on the first request after a quiet period.

11. Screenshots





Home page in the dark theme on a desktop screen (left) and at phone width, 375 px (right). A light theme is also available.